

AWS Floci Cloud Lab

Gobernanza IAM y despliegue continuo con Floci y AWS

Práctica de NextGen Solutions · Partes 1 y 2 · 08/11/2026

Índice

1. AWS Floci Cloud Lab 3
1.1 Resultado 3
1.2 Contenido de la memoria 4
2. Parte 1 · IAM 5
2.1 Entorno 5
2.2 Usuarios y grupos 7
2.3 Políticas 8
2.4 Auditoría con el simulador IAM 11
2.5 Decisiones de seguridad y gobernanza 14
3. Parte 2 · Despliegue 16
3.1 Parte 2: de local a la nube 16
3.2 Ejecución por fases 18
3.3 Comparativa Floci frente a AWS real 23
3.4 Preguntas de la entrega 24
4. Conclusiones 26
4.1 Parte 1 26
4.2 Parte 2 26

1. AWS Floci Cloud Lab

Memoria de las dos partes de la práctica de NextGen Solutions, resueltas con Terraform:

- **Parte 1 · IAM**: estructura de identidades desplegada en Floci. Seis usuarios y tres grupos; los permisos se heredan exclusivamente por pertenencia a grupos. Las decisiones se han comprobado con `simulate-principal-policy`.
- **Parte 2 · Despliegue**: la web de la empresa se publica en un bucket S3. Cada cambio se prueba primero en Floci y GitHub Actions solo lo despliega en AWS (Learner Lab) si la prueba pasa.

1.1 Resultado

Parte 1. El simulador confirmó los permisos previstos para `roberto`, `alicia`, `marta` y `camila`. En particular, `alicia` combina la lectura de auditoría con los permisos de desarrollo, pero `s3:DeleteObject` devuelve `explicitDeny`. Tras parar y reiniciar Floci sin persistencia, la lista de usuarios pasó de seis identidades a cero.

Parte 2. El mismo módulo `s3-website` publicó la web en Floci y en AWS. AWS mostró dos diferencias que Floci no detecta: `BucketAlreadyExists` con el nombre `nextgen` y un 403 mientras la web no tenía permiso público. En GitHub Actions, un cambio correcto llegó solo a producción. Un cambio que eliminaba la portada se detuvo en la prueba en Floci, el despliegue no se ejecutó y la web pública siguió funcionando.

1.2 Contenido de la memoria

Sección	Contenido
Entorno Floci	Preparación, endpoint local y límite del emulador
Flujo de trabajo	Plan y despliegue Terraform, verificación y reinicio
Usuarios y grupos	Identidades, pertenencias y ausencia de políticas directas
Dev-Team	Lectura S3, operaciones DynamoDB y denegación de borrado
Ops-Team	Operaciones EC2 y gestión Lambda
SecOps-Team	Consulta para auditoría y protección frente a operaciones destructivas
Auditoría	Resultados reales del simulador IAM
Decisiones	Seguridad, gobernanza y ciclo de vida efímero
De local a la nube	Arquitectura del despliegue y adaptación del kit a Terraform
Fases	Floci, AWS, GitHub Actions y el error a propósito, con capturas
Comparativa	Floci frente a AWS real
Preguntas	Límites del emulador, valor de CI/CD y gestión de secretos
Conclusiones	Evaluación del diseño y límites

La infraestructura IAM está en `envs/floci/` y `modules/iam-group/`, con las políticas JSON en `policies/`. La de la web está en `envs/web-floci/`, `envs/web-aws/` y `modules/s3-website/`. Los scripts están en `scripts/` y los datos íntegros obtenidos de Floci y de AWS, en `evidence/`.

2. Parte 1 · IAM

2.1 Entorno

Entorno Floci

Herramientas y conexión

La práctica se ejecutó en macOS con Terraform 1.17.0, Docker 28.3.3, AWS CLI 2.37.13 y Floci CLI 0.4.0. Floci arrancó un contenedor local accesible en `http://localhost:4566`. El proveedor AWS de Terraform usa credenciales ficticias `test/test` y dirige IAM y STS al endpoint local. Los scripts de verificación fijan también ese endpoint.

```
docker info
terraform version
aws --version
floci doctor
floci start
eval "$(floci env)"
aws iam list-users
```

`floci env` afecta solo a la terminal actual. Terraform no depende de esas variables: su proveedor declara el endpoint local y credenciales de prueba en `envs/floci/providers.tf`. El verificador usa `--endpoint-url http://localhost:4566` para evitar que una configuración AWS personal redirija las llamadas a la nube real.

Limitación de IAM en el emulador

El enunciado indica que Floci almacena políticas IAM, pero no aplica por defecto sus restricciones a las peticiones de servicio. Una operación de S3 hecha con las credenciales ficticias no demuestra el permiso efectivo de `roberto`. La evidencia de autorización procede de `aws iam simulate-principal-policy`, que evalúa las políticas de sus grupos.

Persistencia

La ejecución normal se hizo sin `--persist`. Al detener y arrancar el emulador se eliminó el contenedor anterior y desaparecieron las seis identidades. La salida íntegra está en `evidence/ephemeral.json`: `users_before` contiene los seis nombres y `users_after` es una lista vacía. El estado local de Terraform permanece, por lo que un nuevo `terraform apply` detecta los recursos ausentes y los reconstruye. Para conservar datos en Floci entre arranques se necesitaría configurar persistencia, lo que no forma parte de esta prueba.

Flujo de trabajo con Terraform

El entorno `envs/floci/` usa el proveedor oficial `hashicorp/aws` con credenciales ficticias y el endpoint IAM local. `modules/iam-group/` crea un grupo y le asocia su política inline JSON. En `locals.tf` se declaran los seis usuarios y sus pertenencias; `main.tf` crea los recursos. El archivo `.terraform.lock.hcl` fija la versión exacta del proveedor para repetir el despliegue.

Despliegue

Desde la raíz del repositorio, con Docker Desktop en funcionamiento:

```
floci start
terraform -chdir=envs/floci init
terraform -chdir=envs/floci fmt -check
terraform -chdir=envs/floci validate
terraform -chdir=envs/floci test
terraform -chdir=envs/floci plan
terraform -chdir=envs/floci apply
python3 scripts/verify.py
```

`bash scripts/deploy.sh` agrupa el `init` y el `apply` para repetir la creación. El primer plan mostró 18 recursos: seis usuarios, tres grupos, tres políticas inline y seis recursos de pertenencia (uno por usuario; `alicia` tiene dos grupos). Tras aplicar, un segundo `terraform plan -detailed-exitcode` terminó con «No changes».

`evidence/terraform-state.txt` lista los 18 recursos registrados en el estado de Terraform tras la reconstrucción.

`verify.py` contrasta el estado real con los JSON del repositorio y el simulador IAM. Guarda respuestas completas en `evidence/verification.json` y la comparación en `evidence/verification.md`. Para comprobar manualmente a `alicia`:

```
eval "$(floci env)"
aws iam list-groups-for-user --user-name alicia
arn="$(aws iam get-user --user-name alicia --query User.Arn --output text)"
aws iam simulate-principal-policy \
  --policy-source-arn "$arn" \
  --action-names s3:GetObject s3:DeleteObject cloudtrail:LookupEvents \
  --query 'EvaluationResults[].[EvalActionName,EvalDecision]' \
  --output table
```

Prueba de entorno efímero

Después de guardar las simulaciones, `bash scripts/check-ephemeral.sh` comprueba que el reinicio de Floci borra las identidades. `terraform -chdir=envs/floci plan` detecta los 18 recursos ausentes; `bash scripts/deploy.sh` los recrea y `python3 scripts/verify.py` vuelve a comprobarlos. El resultado del reinicio queda en `evidence/ephemeral.json`.

2.2 Usuarios y grupos

Grupo	Usuarios	Función
Dev-Team	roberto, sandra, alicia	Desarrollo sobre S3 y DynamoDB
Ops-Team	marta, alexis	Operación EC2 y Lambda
SecOps-Team	camila, alicia	Consulta y auditoría

Terraform crea los seis usuarios con `aws_iam_user`, los tres grupos con el módulo `iam-group` y las siete pertenencias con `aws_iam_user_group_membership`. La verificación contra Floci confirmó esas identidades y pertenencias. `alicia` aparece tanto en Dev-Team como en SecOps-Team. Cada grupo tiene una política inline propia. Se consultaron `list-user-policies` y `list-attached-user-policies` para los seis usuarios; ambas listas estaban vacías en todos los casos. Así se confirma que ningún permiso se asignó directamente a una persona.

La estructura permite cambiar permisos una sola vez en la política del grupo. Cuando alguien cambia de función, se modifica su pertenencia en vez de editar políticas individuales. La doble pertenencia de `alicia` es deliberada y se analiza en Decisiones.

2.3 Políticas

Política de Dev-Team

El archivo `policies/dev-team-policy.json` sigue el modelo del enunciado. Terraform lo lee desde `envs/floci/main.tf` y el módulo `iam-group` lo asocia al grupo como `DevTeamPolicy`.

Bloque	Efecto
`LecturaS3`	Permite `GetObject`, `ListBucket` y `ListAllMyBuckets`.
`CrudDynamoDB`	Permite leer, escribir, actualizar, borrar elementos, consultar y escanear tablas. No permite borrar tablas.
`DenegarBorradoS3`	Deniega explícitamente `DeleteObject` y `DeleteBucket` en todos los recursos.

En la simulación de `roberto`, `s3:GetObject` y `dynamodb:PutItem` resultaron `allowed`; `s3:DeleteObject` resultó `explicitDeny`. `ec2:StartInstances` resultó `implicitDeny`, lo que confirma que Dev-Team no recibe permisos de operación EC2.

La denegación explícita protege también a `alicia`, que pertenece a dos grupos. Si otro grupo llegara a permitir `s3:*`, el `Deny` seguiría prevaleciendo sobre ese `Allow`.

Política de Ops-Team

Terraform lee `policies/ops-team-policy.json` y el módulo `iam-group` la asocia a Ops-Team como `OpsTeamPolicy`.

policies/ops-team-policy.json

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ConsultarEC2",
      "Effect": "Allow",
      "Action": "ec2:Describe*",
      "Resource": "*"
    },
    {
      "Sid": "ArrancarYPararEC2",
      "Effect": "Allow",
      "Action": ["ec2:StartInstances", "ec2:StopInstances"],
      "Resource": "*"
    },
    {
      "Sid": "GestionarLambda",
      "Effect": "Allow",
      "Action": [
        "lambda:ListFunctions",
        "lambda:GetFunction",
        "lambda:CreateFunction",
        "lambda:UpdateFunctionCode",
        "lambda:UpdateFunctionConfiguration",
        "lambda:DeleteFunction",
        "lambda:InvokeFunction"
      ],
      "Resource": "*"
    }
  ]
}
```

Bloque	Efecto
`ConsultarEC2`	`ec2:Describe*` permite consultar instancias y otros datos EC2.
`ArrancarYPararEC2`	Permite `StartInstances` y `StopInstances`, sin conceder terminación.
`GestionarLambda`	Permite listar, consultar, crear, actualizar, borrar e invocar funciones Lambda mediante acciones enumeradas.

En la simulación de `marta`, `ec2:DescribeInstances`, `ec2:StartInstances`, `ec2:StopInstances`, `lambda:ListFunctions` y `lambda:CreateFunction` resultaron `allowed`. `s3:GetObject` dio `implicitDeny`.

Se enumeran las acciones de Lambda en vez de usar `lambda:*`, para evitar permisos adicionales no exigidos. El emulador usa `Resource: "*"` porque el enunciado no define ARNs concretos; en una cuenta real se acotarían instancias y funciones por ARN y se revisarían dependencias como `iam:PassRole` para crear funciones con un rol de ejecución.

Política de SecOps-Team

Terraform lee `policies/secops-team-policy.json` y el módulo `iam-group` la asocia a SecOps-Team como `SecOpsTeamPolicy`.

policies/secops-team-policy.json

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ConsultarCloudTrail",
      "Effect": "Allow",
      "Action": ["cloudtrail:LookupEvents", "cloudtrail:Describe*", "cloudtrail:Get*"],
      "Resource": "*"
    },
    {
      "Sid": "ConsultarCloudWatchLogs",
      "Effect": "Allow",
      "Action": ["logs:Describe*", "logs:Get*", "logs:FilterLogEvents"],
      "Resource": "*"
    },
    {
      "Sid": "ConsultarIAM",
      "Effect": "Allow",
      "Action": ["iam:Get*", "iam:List*"],
      "Resource": "*"
    },
    {
      "Sid": "ProtegerRecursosProduccion",
      "Effect": "Deny",
      "Action": ["ec2:TerminateInstances", "s3:DeleteObject", "dynamodb:DeleteTable"],
      "Resource": "*"
    }
  ]
}
```

Bloque	Efecto
`ConsultarCloudTrail`	Permite `LookupEvents`, `Describe*` y `Get*` de CloudTrail.
`ConsultarCloudWatchLogs`	Permite `Describe*`, `Get*` y `FilterLogEvents` de CloudWatch Logs.
`ConsultarIAM`	Permite `Get*` y `List*` de IAM para inspeccionar la configuración.
`ProtegerRecursosProduccion`	Deniega `ec2:TerminateInstances`, `s3:DeleteObject` y `dynamodb:DeleteTable`.

El `Deny` se aplica a `Resource: "*"`: protege también los recursos de producción y, de forma deliberadamente más estricta, impide esas tres operaciones sobre cualquier recurso. Así seguirá prevaleciendo si en el futuro otro grupo concede permisos amplios. Para limitarlo solo a producción en AWS real harían falta nombres o etiquetas de recursos definidos y condiciones adecuadas a cada servicio.

2.4 Auditoría con el simulador IAM

Terraform creó los usuarios, grupos, pertenencias y políticas en Floci. El enunciado advierte que Floci no aplica por defecto las políticas al invocar los servicios. Por eso se usó `aws iam simulate-principal-policy` sobre el ARN de cada usuario. El simulador incluyó las políticas de todos sus grupos; no ejecutó las acciones sobre recursos reales.

Resultados observados (selección)

Usuario	Acción	Decisión
roberto	`s3:GetObject`	`allowed`
roberto	`s3:DeleteObject`	`explicitDeny`
roberto	`dynamodb:PutItem`	`allowed`
alicia	`s3:GetObject`	`allowed`
alicia	`s3:DeleteObject`	`explicitDeny`
alicia	`cloudtrail:LookupEvents`	`allowed`
marta	`ec2:DescribeInstances`	`allowed`
marta	`ec2:StartInstances`	`allowed`
marta	`lambda:CreateFunction`	`allowed`
marta	`s3:GetObject`	`implicitDeny`
camila	`cloudtrail:LookupEvents`	`allowed`
camila	`logs:FilterLogEvents`	`allowed`
camila	`s3:DeleteObject`	`explicitDeny`
camila	`dynamodb:PutItem`	`implicitDeny`

Estos resultados se obtuvieron el 30 de octubre de 2026. El archivo `evidence/verification.json` conserva la respuesta íntegra de Floci, incluidas las evaluaciones por acción; `evidence/verification.md` presenta la comparación entre valor esperado y obtenido. La comprobación automatizada también verificó las siete pertenencias, la coincidencia de las tres políticas aplicadas con los JSON del repositorio y la ausencia de políticas directas en usuarios.

`allowed` significa que existe un `Allow` aplicable; `implicitDeny`, que ninguna política concede la acción; `explicitDeny`, que una regla `Deny` aplicable prevalece sobre cualquier permiso concedido.

```
Floci IAM simulator  |  roberto
ARN: arn:aws:iam::000000000000:user/roberto
Fecha UTC: 2026-10-30T10:35:53.833394+00:00

ACCION                          DECISION
s3:GetObject                    allowed
s3:DeleteObject                 explicitDeny
dynamodb:PutItem                allowed
ec2:StartInstances              implicitDeny
```

```
Floci IAM simulator  |  alicia
ARN: arn:aws:iam::000000000000:user/alicia
Fecha UTC: 2026-10-30T10:35:53.833394+00:00

ACCION                          DECISION
s3:GetObject                    allowed
s3:DeleteObject                 explicitDeny
dynamodb:PutItem                allowed
cloudtrail:LookupEvents         allowed
iam:ListUsers                   allowed
ec2:TerminateInstances          explicitDeny
dynamodb:DeleteTable            explicitDeny
```

```
Floci IAM simulator  |  marta
ARN: arn:aws:iam::000000000000:user/marta
Fecha UTC: 2026-10-30T10:35:53.833394+00:00

ACCION                          DECISION
ec2:DescribeInstances           allowed
ec2:StartInstances              allowed
ec2:StopInstances               allowed
lambda:ListFunctions            allowed
lambda:CreateFunction           allowed
s3:GetObject                    implicitDeny
```

```
Floci IAM simulator  |  camila
ARN: arn:aws:iam::000000000000:user/camila
Fecha UTC: 2026-10-30T10:35:53.833394+00:00

ACCION                          DECISION
cloudtrail:LookupEvents         allowed
logs:FilterLogEvents            allowed
iam:ListUsers                   allowed
s3:DeleteObject                 explicitDeny
ec2:TerminateInstances          explicitDeny
dynamodb:PutItem                implicitDeny
```

2.5 Decisiones de seguridad y gobernanza

Por qué Terraform

La configuración de `envs/floci/` declara de forma revisable las identidades, sus pertenencias y las políticas de grupo. El módulo `modules/iam-group/` evita repetir la creación de cada grupo y la asociación de su JSON. `terraform plan` muestra el cambio antes de aplicarlo; tras el primer despliegue, el plan devolvió «No changes». Esto permite reconstruir exactamente el laboratorio después de un reinicio efímero de Floci. El estado Terraform se guarda localmente y se excluye de Git; `.terraform.lock.hcl` fija la versión del proveedor.

Cómo se interpreta cada política JSON

`Version` identifica la versión del lenguaje de políticas de IAM, aquí `2012-10-17`; no es la fecha de creación del archivo. `Statement` contiene las reglas evaluables. `Sid` da un nombre a cada regla para reconocer su intención. `Effect` indica `Allow` o `Deny`. `Action` enumera operaciones AWS, con comodines cuando se pide una familia de acciones. `Resource` determina a qué recursos se aplica cada regla.

En esta práctica el enunciado no proporciona ARNs de buckets, tablas, instancias o funciones; por eso las políticas usan `Resource: "*"`. El mínimo privilegio se aproxima restringiendo las acciones por grupo. Antes de pasar el diseño a una cuenta AWS real habría que acotar recursos y revisar cada comodín.

Por qué se usa un Deny explícito

La ausencia de `s3:DeleteObject` en los `Allow` de Dev-Team produciría solo `implicitDeny`. Si en el futuro otra política de cualquiera de los grupos de `alicia` concediera `s3:*`, ese permiso permitiría borrar objetos. En cambio, el `Deny` explícito de Dev-Team prevalece aunque aparezca otro `Allow`. La simulación actual de `alicia` confirmó `explicitDeny` para `s3:DeleteObject`.

SecOps-Team también deniega explícitamente tres operaciones destructivas. Como `alicia` pertenece a ese grupo, recibe ambas protecciones. El bloqueo de SecOps usa `Resource: "*"`, más amplio que «producción»; se eligió así porque el ejercicio no identifica recursos de producción concretos.

Grupos frente a permisos individuales

Con grupos, una política expresa la función de un equipo y se modifica una sola vez. Incorporar a una persona consiste en añadirla al grupo; retirar una función consiste en quitar esa

pertenencia. Esto facilita la revisión de quién tiene cada capacidad y reduce excepciones difíciles de encontrar.

Con permisos asignados usuario por usuario, un cambio de función obliga a localizar y editar cada identidad. Para 10 empleados todavía podría revisarse manualmente; con 100 aumenta la probabilidad de diferencias y permisos olvidados; con 1000 el coste operativo y de auditoría hace necesario automatizar las altas, bajas y revisiones sobre grupos o roles. La doble pertenencia de `alicia` muestra cómo combinar funciones sin duplicar políticas.

Infraestructura efímera para CI/CD

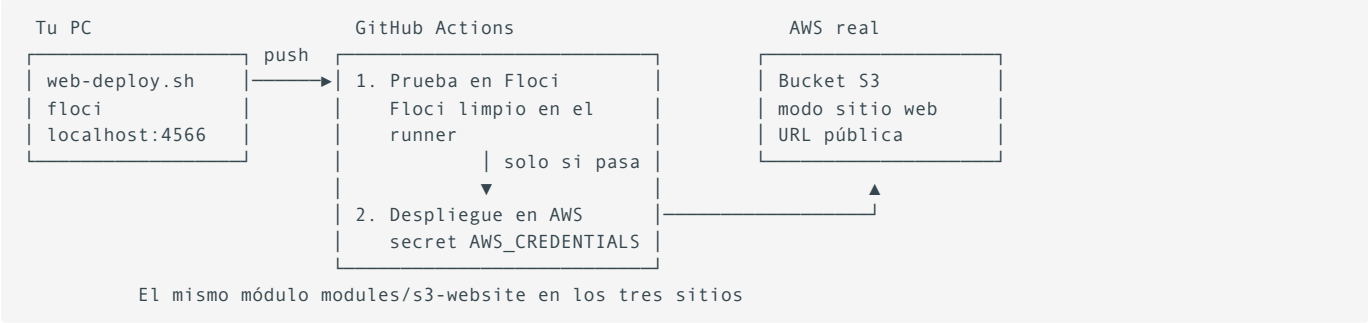
La comprobación real registró seis usuarios creados por Terraform antes de `floci stop` y ninguno después de `floci start`, sin persistencia configurada. El estado local de Terraform permaneció; el plan detectó 18 recursos ausentes y `apply` los recreó. Cada ejecución de una tubería CI/CD puede partir de un estado vacío, aplicar la configuración, probarla y descartarla al finalizar. Esto evita que un test dependa de datos residuales de otro y hace reproducible el entorno. Para conservar estado en Floci entre arranques, habría que elegir explícitamente un modo persistente.

3. Parte 2 · Despliegue

3.1 Parte 2: de local a la nube

NextGen Solutions quiere publicar su web sin riesgos: cada cambio se prueba primero en Floci y solo llega a AWS si la prueba pasa. El profesorado entregó un kit con la web, un `deploy.sh` en bash basado en la AWS CLI y un workflow de GitHub Actions. En este repositorio, el despliegue se ha llevado a Terraform para seguir el mismo enfoque que la Parte 1: un módulo reutilizable y un entorno por destino.

Arquitectura



Pieza	Equivalente en el kit	Función
`web/`	`web/`	La web, sin cambios salvo el nombre del pie
`config/web.tfvars`	`config.sh`	`bucket_name` y `public_read`; es lo único que se configura
`modules/s3-website/`	Pasos 2 a 4 de `deploy.sh`	Modo sitio web, bloqueo de acceso público, política de lectura pública y archivos
`envs/web-floci/`	`usar-floci.sh`	Endpoint `localhost:4566`, credenciales `test` y estado local
`envs/web-aws/`	`usar-aws.sh`	AWS real, credenciales de Learner Lab y estado remoto en S3
`scripts/web-deploy.sh`	`deploy.sh`	Crea el bucket, aplica Terraform y comprueba que la web responde 200
`.github/workflows/web.yml`	`.github/workflows/deploy.yml`	Prueba en Floci y, solo si pasa, despliegue en AWS

```
bash scripts/web-deploy.sh floci          # Fase 1
bash scripts/web-deploy.sh aws            # Fase 2
bash scripts/web-deploy.sh aws destroy    # Limpieza
```

Decisiones de diseño

Un módulo, dos entornos. El módulo no sabe dónde se despliega. `envs/web-floci` y `envs/web-aws` solo difieren en el proveedor (endpoint y credenciales) y en el lugar donde se guarda el estado. Así se mantiene la idea del enunciado: el mismo código en los tres sitios.

Estado remoto en AWS. Cada ejecución de GitHub Actions arranca en un equipo limpio. Con un estado local, el segundo despliegue no sabría que la web ya existe e intentaría crearla de nuevo. `envs/web-aws` guarda el estado en un bucket `nextgen-tfstate-<cuenta>`, con versionado y bloqueo mediante `use_lockfile`; el script lo crea la primera vez. En Floci el estado local es correcto: interesa partir siempre de un entorno vacío.

Validación antes de desplegar. El módulo exige un nombre de bucket válido y que existan `index.html` y `error.html`. Una web sin portada se detiene en el `plan`, antes de tocar ningún recurso.

El bucket lo crea el script. En Learner Lab, una política de control de servicio (SCP) de la organización deniega `s3:GetBucketObjectLockConfiguration`. El recurso `aws_s3_bucket` del proveedor de Terraform lee siempre esa configuración después de crear el bucket, y en AWS real trata la denegación como un error fatal. El primer intento dejó el bucket creado pero marcado como *tainted* (evidencia). Se comprobaron las demás lecturas del bucket y solo esa estaba denegada. Por eso `web-deploy.sh` crea el bucket con la AWS CLI, como el paso 1/4 del kit, y Terraform gestiona todo lo que va dentro.

Contenido

Página	Contenido
Fases	Ejecución de las fases 1 a 4 con sus resultados y capturas
Comparativa	Tabla Floci frente a AWS real
Preguntas	Límites del emulador, valor de CI/CD y gestión de secretos

3.2 Ejecución por fases

Todas las salidas proceden de ejecuciones reales del 8 de noviembre de 2026. Los logs completos están en `evidence/parte2/`. En los errores de AWS se han ocultado los identificadores de cuenta y de organización.

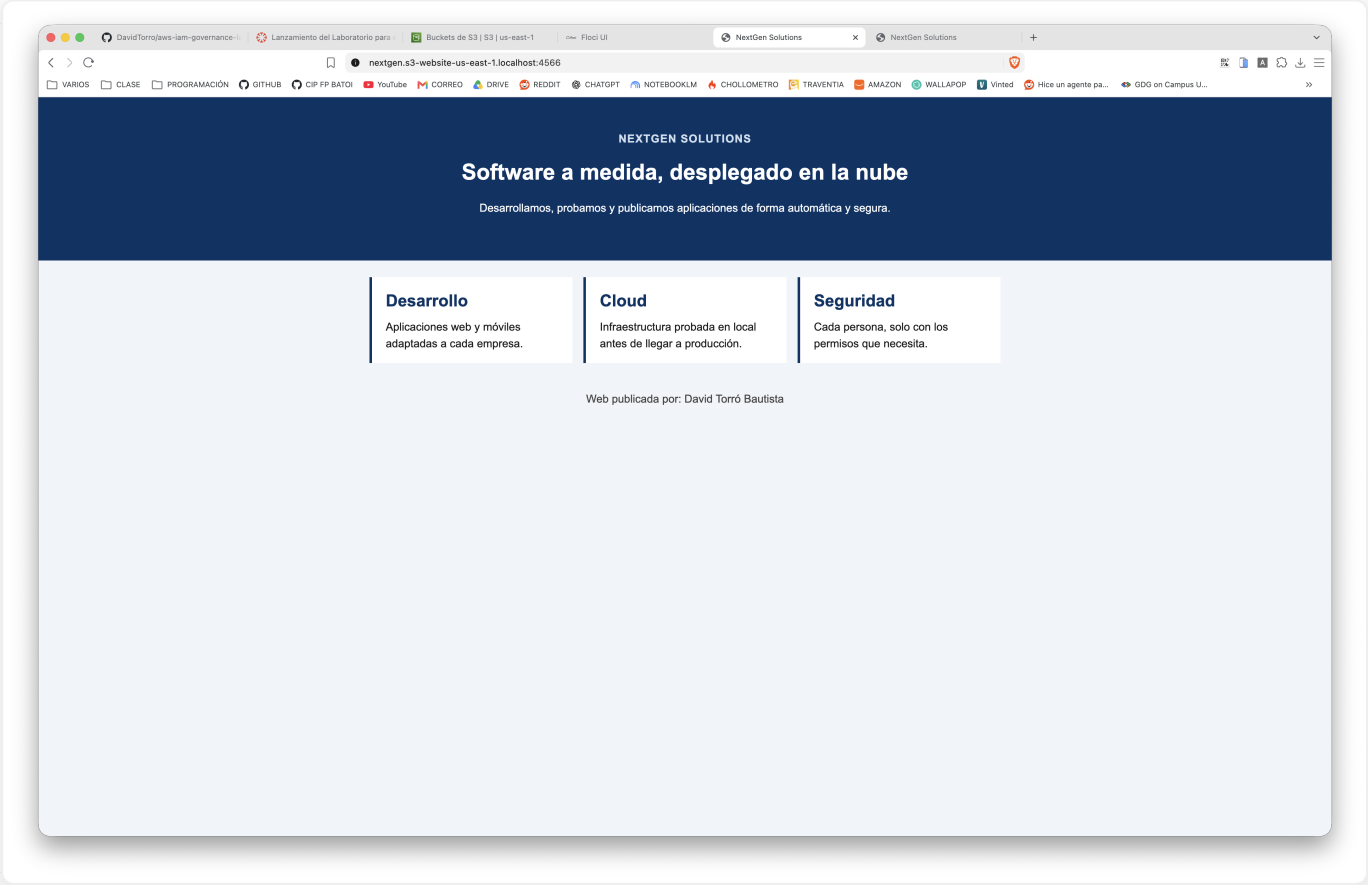
Fase 1: la web en el PC

Con `bucket_name = "nextgen"` y `public_read = false`:

```
floci start
bash scripts/web-deploy.sh floci
```

```
== Destino: FLOCI (http://localhost:4566) ==
Bucket: nextgen
make_bucket: nextgen
Apply complete! Resources: 5 added, 0 changed, 0 destroyed.
OK. Tu web está en: http://nextgen.s3-website-us-east-1.localhost:4566/
```

Terraform creó la configuración de sitio web, el bloqueo de acceso público y los tres archivos. Un segundo `terraform plan` no detectó cambios. La web respondió 200 aunque era privada y tenía bloqueado el acceso público. La misma dirección no carga en el móvil: `localhost` significa «este mismo dispositivo» y Floci solo existe en el PC.



Fase 2: el salto a AWS

Con las credenciales de Learner Lab en `~/.aws/credentials`, se lanzó exactamente el mismo script contra AWS:

```
bash scripts/web-deploy.sh aws
```

Error 1: el nombre ya existe. Con `bucket_name = "nextgen"`:

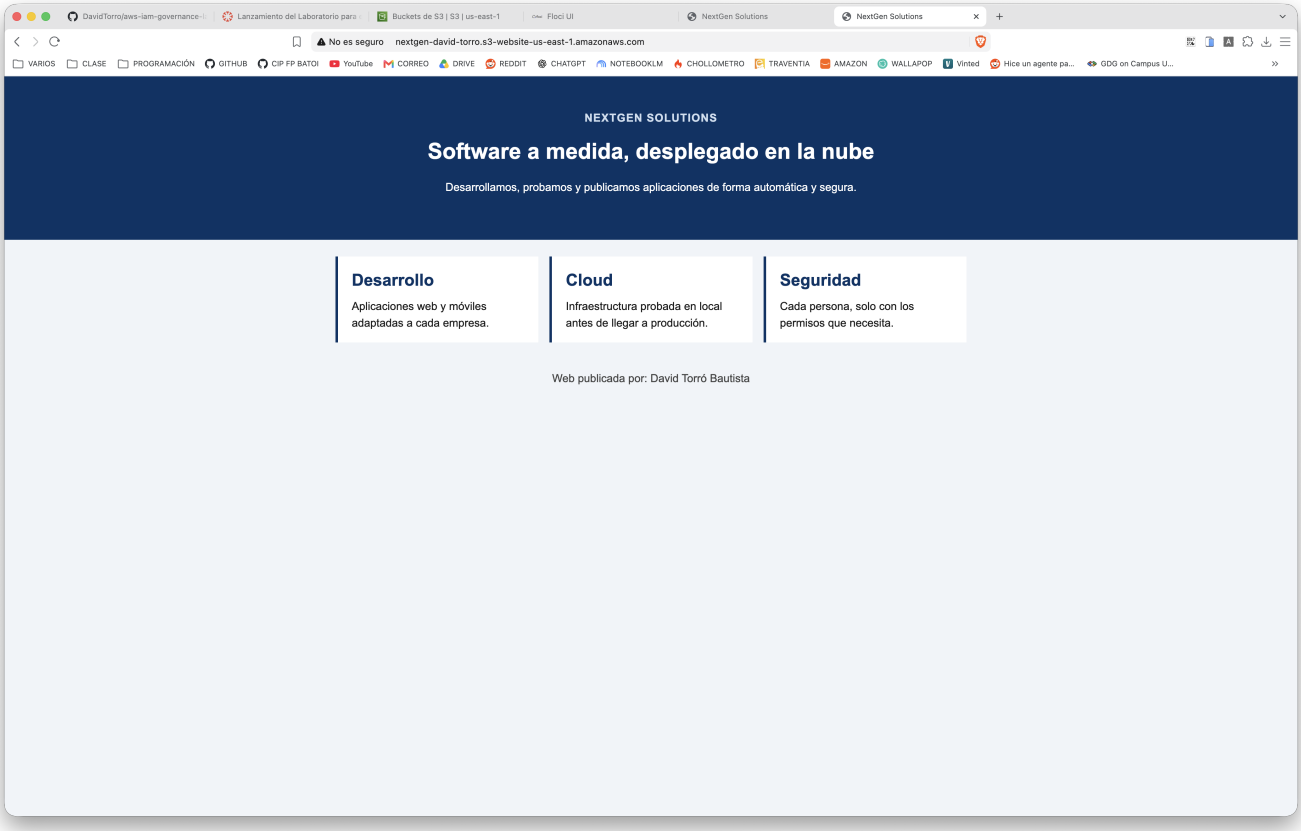
```
make_bucket failed: s3://nextgen An error occurred (BucketAlreadyExists) when calling
the CreateBucket operation: The requested bucket name is not available. The bucket
namespace is shared by all users of the system. Please select a different name and try again.
```

Error 2: la web es privada. Con `bucket_name = "nextgen-david-torro"`, Terraform aplicó los cambios sin errores, pero la comprobación HTTP falló:

```
Apply complete! Resources: 5 added, 0 changed, 0 destroyed.
Comprobando que la web responde...
ERROR: la web responde con el código 403 (se esperaba 200).
```

Solución. Con `public_read = true`, Terraform relajó el bloqueo de políticas públicas y añadió la política `LecturaPublica` (`s3:GetObject` para `*`). Las ACL públicas siguen bloqueadas.

```
Plan: 1 to add, 1 to change, 0 to destroy.
Apply complete! Resources: 1 added, 1 changed, 0 destroyed.
OK. Tu web está en: http://nextgen-david-torro.s3-website-us-east-1.amazonaws.com/
```



Fase 3: automatizar con GitHub

Se guardó el bloque de credenciales de Learner Lab como secret `AWS_CREDENTIALS` del repositorio. El código entró en `main` con la PR #5, y ese merge lanzó el workflow `Web`.

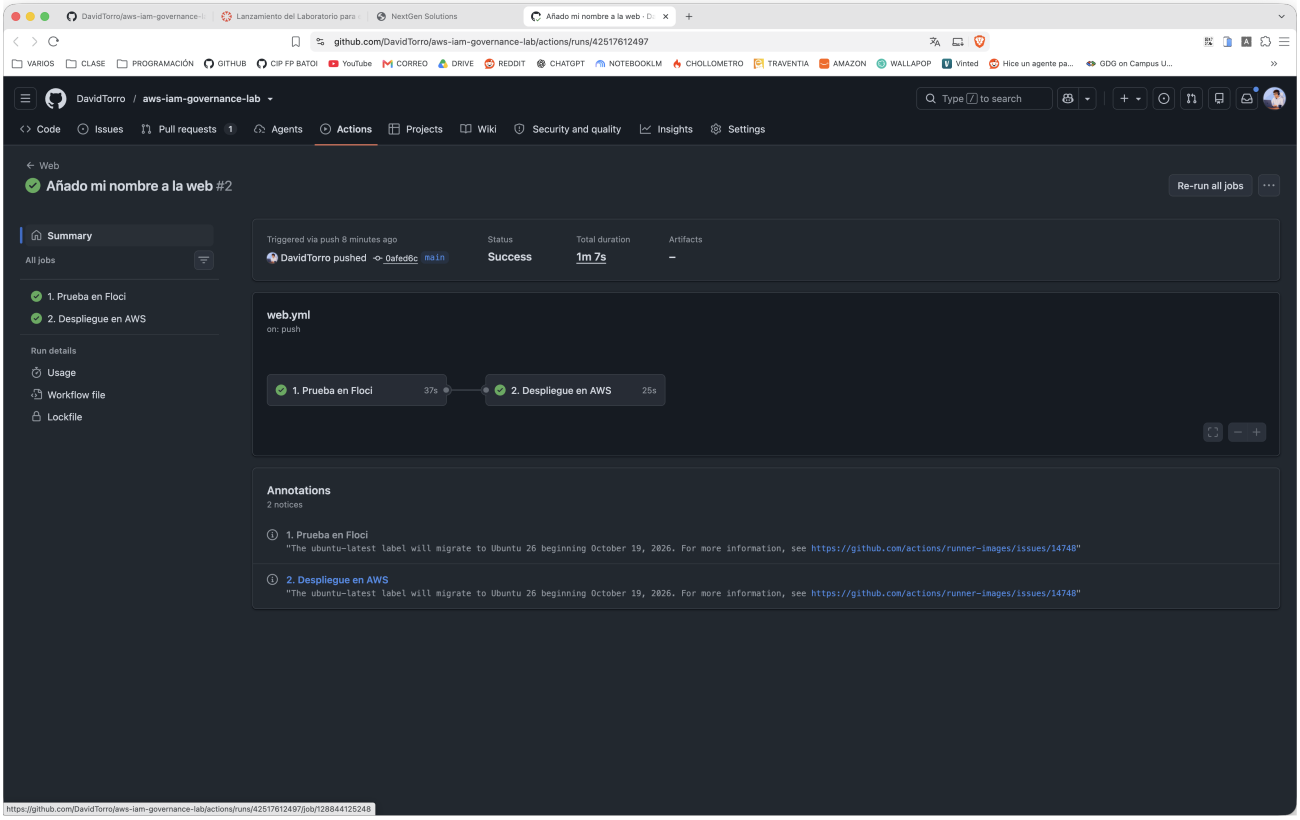
Los dos primeros intentos del trabajo de AWS fallaron con `AccessDenied … policy/voc-cancel-cred`. El laboratorio estaba parado, y Learner Lab asigna esa política al rol para denegarlo todo. Tras pulsar *Start Lab* y actualizar el secret, el tercer intento terminó en verde. El trabajo de AWS encontró el estado remoto creado en la Fase 2 y solo actualizó `index.html`:

```
1. Prueba en Floci    Apply complete! Resources: 6 added, 0 changed, 0 destroyed.
2. Despliegue en AWS  Apply complete! Resources: 0 added, 1 changed, 0 destroyed.
```

Nadie tocó la nube a mano. GitHub creó un equipo temporal, arrancó un Floci vacío, probó la web y después ejecutó el mismo módulo contra AWS con el secret.

Fase 4: cambiar y romper

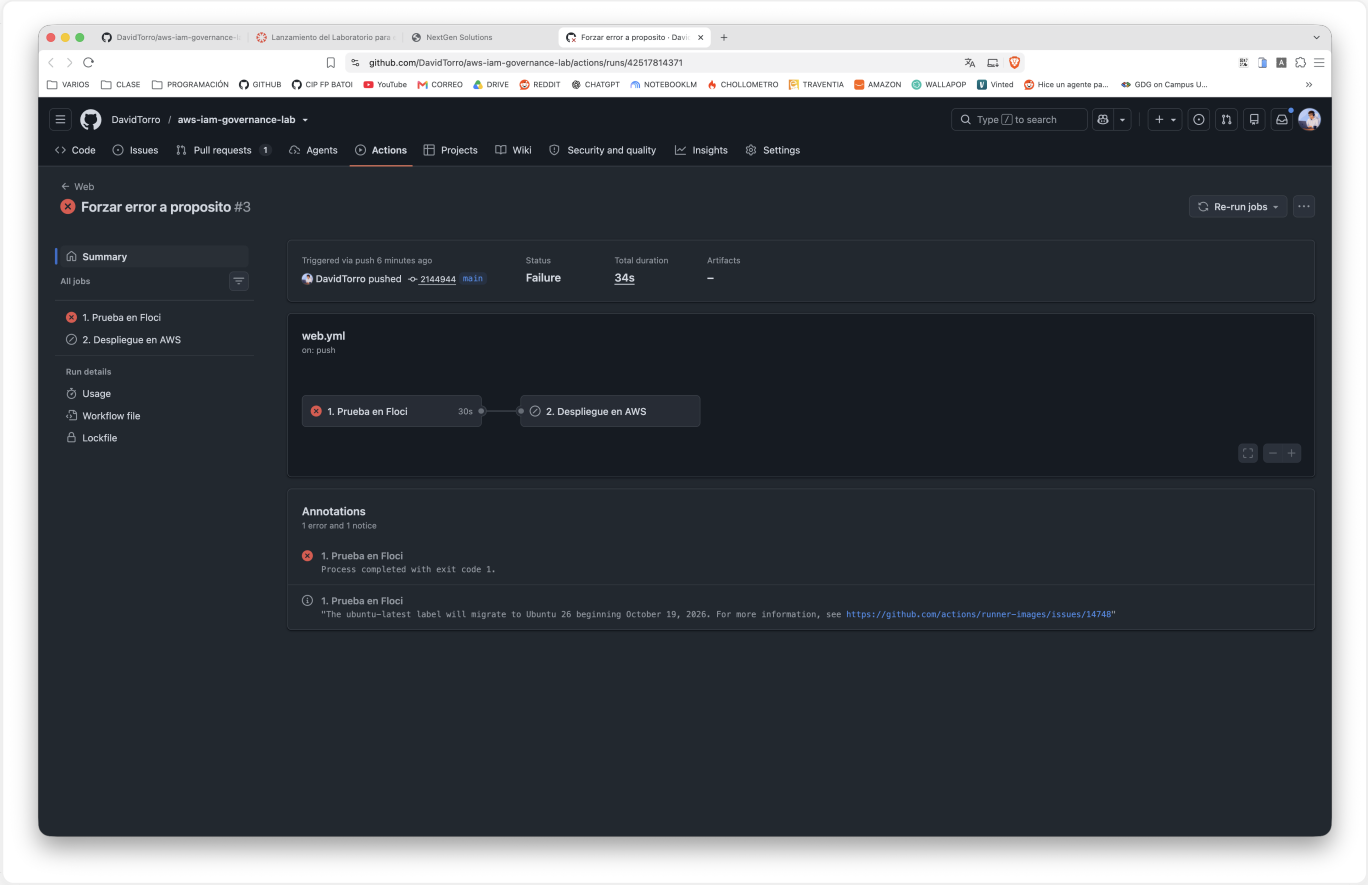
Un cambio correcto llega solo a producción. El commit `0afed6c` sustituyó «ESCRIBE AQUÍ TU NOMBRE» por el nombre del autor. Los dos trabajos terminaron en verde y la web pública mostró el cambio.



Un cambio erróneo no llega a producción. El commit `2144944` renombró `web/index.html` a `web/inicio.html`. La prueba en Floci se detuvo en el `plan`:

```
Error: Invalid value for variable
La web no tiene portada: falta index.html.
```

El despliegue en AWS quedó en `skipped` y la web pública siguió mostrando la versión anterior. El workflow `CI` falló en el mismo commit, porque los tests del módulo también comprueban la carpeta `web/`. El commit `02c17f6` revirtió el error y los dos trabajos volvieron a terminar en verde.



Commit	Prueba en Floci	Despliegue en AWS
`344908a` Merge de la PR #5	✅	✅ (intento 3)
`0afed6c` Añado mi nombre	✅	✅
`2144944` Error a propósito	❌	⏭️ sin ejecutar
`02c17f6` Revert	✅	✅

El detalle de cada ejecución está en `evidence/parte2/actions.md`.

Limpieza

```
bash scripts/web-deploy.sh aws destroy
floci stop
```

`destroy` elimina con Terraform la política, el bloqueo de acceso público, la configuración de sitio web y los archivos, y después borra el bucket con `aws s3 rb --force`. El bucket del estado remoto se borra al final a mano. Por último se pulsa *End Lab* en Learner Lab.

3.3 Comparativa Floci frente a AWS real

	Floci	**AWS real**
¿Desde dónde se puede ver la web?	Solo desde el PC que ejecuta Floci: `http://nextgen.s3-website-us-east-1.localhost:4566/`. En el móvil no carga, porque `localhost` es el propio dispositivo.	Desde cualquier dispositivo con Internet: `http://nextgen-david-torro.s3-website-us-east-1.amazonaws.com/`.
¿Se puede usar el nombre `nextgen`?	Sí. El emulador es una nube privada de una sola cuenta y nadie más usa ese nombre.	No. `BucketAlreadyExists`: el espacio de nombres de S3 es global y otra cuenta ya tiene `nextgen`.
¿Se puede leer la web sin dar permiso público?	Sí. Responde 200 con `public_read = false` y todo el acceso público bloqueado: Floci no aplica esa restricción.	No. Responde 403 hasta que se permite la política pública y se añade `LecturaPublica`.
¿Qué credenciales se usan?	Ficticias (`test` / `test`), fijadas en `envs/web-floci/providers.tf`.	Temporales de Learner Lab (clave, secreto y token de sesión, unas 4 horas): en `~/.aws/credentials` en el PC y en el secret `AWS_CREDENTIALS` en GitHub Actions.
¿Qué cambia en `deploy.sh` para usarlo aquí?	Nada del código. Se ejecuta `bash scripts/web-deploy.sh floci`, que usa `envs/web-floci`: endpoint `localhost:4566`, credenciales de prueba y estado local.	Nada del código. Se ejecuta `bash scripts/web-deploy.sh aws`, que usa `envs/web-aws`: sin endpoint (AWS por defecto), credenciales reales y estado remoto en S3.

El módulo `modules/s3-website`, la web y `config/web.tfvars` son idénticos en los dos destinos. Solo cambian tres cosas: el endpoint, las credenciales y el lugar donde se guarda el estado. Es el equivalente de las dos variables del kit, `AWS_ENDPOINT_URL` y las credenciales, más el estado, que Terraform necesita.

3.4 Preguntas de la entrega

Los límites del emulador

Los dos errores de la Fase 2 fueron:

```
make_bucket failed: s3://nextgen An error occurred (BucketAlreadyExists) when calling
the CreateBucket operation: The requested bucket name is not available. The bucket
namespace is shared by all users of the system. Please select a different name and try again.
```

```
ERROR: la web responde con el código 403 (se esperaba 200).
```

¿Por qué no aparecieron en Floci? Floci emula una única cuenta en el PC. No existe un espacio de nombres compartido con otros usuarios, así que `nextgen` estaba libre. Además, Floci no aplica las restricciones de acceso de S3 al servir la web: el bloqueo de acceso público y la ausencia de política no impidieron leerla. En la Parte 1 ya se vio lo mismo con IAM: Floci almacena las políticas, pero no las aplica.

Learner Lab añadió dos diferencias más que Floci tampoco podía mostrar. Una SCP de la organización deniega `s3:GetBucketObjectLockConfiguration`, lo que impide usar el recurso `aws_s3_bucket` de Terraform. Y cuando el laboratorio está parado, la política `voc-cancel-cred` deniega todas las acciones.

¿Probar en Floci garantiza que funcionará en AWS? No. Floci reproduce las API, pero no las reglas de la cuenta real: unicidad global de nombres, permisos efectivos, políticas de la organización, caducidad de credenciales y presupuesto.

¿Para qué sirve, si no lo garantiza? Para descartar, gratis y sin riesgo, los errores que dependen de nuestro código. En la Fase 4 detectó una web sin portada antes de que llegara a producción. También comprueba que Terraform crea y actualiza los recursos y que la web responde. Cada prueba parte de un entorno vacío, así que no depende de restos de ejecuciones anteriores. Los fallos que sí llegan a AWS son de entorno, no de código, y son menos y más fáciles de diagnosticar. Floci es un filtro rápido, no un sustituto de AWS.

El valor de CI/CD

¿Qué le pasó a la web pública cuando se subió el error? Nada. La prueba en Floci falló en el `plan` («La web no tiene portada: falta index.html.»), el trabajo de AWS quedó en `skipped` y la web siguió sirviendo la versión anterior con el nombre del autor. Tras `git revert`, los dos trabajos volvieron a terminar en verde.

¿Qué habría pasado sin el trabajo de prueba? Con el `deploy.sh` del kit, el despliegue habría ejecutado `aws s3 sync --delete`: habría borrado `index.html` del bucket y subido `inicio.html`. Desde ese momento, todos los clientes que abrieran la web habrían recibido la página de error

en lugar de la portada, hasta que alguien lo detectara y lo arreglara a mano. En una empresa con miles de clientes, eso supone una caída visible, pérdida de ventas y de confianza, y una corrección con prisas sobre producción.

En esta implementación hay además una segunda barrera: la validación del módulo detendría también el `plan` contra AWS. Aun así, el trabajo de prueba sigue siendo necesario para los errores que una validación no puede prever, porque ejecuta el despliegue completo y comprueba que la web responde.

Gestión de secretos

¿Por qué las credenciales están en un secret de GitHub y no en el código? El repositorio es público y Git conserva todo el historial. Una clave escrita en `web-deploy.sh` o en `web.tfvars` quedaría a la vista de cualquiera, y borrarla después no la quita de los commits anteriores. Un secret de GitHub, en cambio:

- se guarda cifrado y ni siquiera el propietario puede volver a leerlo;
- solo llega al trabajo que lo pide (`2. Despliegue en AWS`) y aparece enmascarado como `***` en los logs;
- no se entrega a workflows lanzados desde forks;
- se puede cambiar sin tocar el código, como se hizo en la Fase 3 al reiniciar el laboratorio.

En el runner, las credenciales solo existen en `~/.aws/credentials` de una máquina temporal que se destruye al terminar.

¿Qué podría ocurrir si se subieran a un repositorio público? Hay bots que rastrean GitHub en busca de claves y las encuentran en minutos. Con ellas, cualquiera podría actuar como el propietario de la cuenta: crear recursos caros (por ejemplo, instancias para minar criptomonedas), borrar o modificar la web y los datos, leer información privada o usar la cuenta para atacar a terceros. El coste y la responsabilidad recaerían en el propietario. Habría que revocar las claves de inmediato y limpiar el historial. Las credenciales de Learner Lab caducan en unas 4 horas y la cuenta tiene un presupuesto limitado, lo que reduce el daño, pero no lo elimina.

Una observación relacionada: los mensajes `AccessDenied` de AWS incluyen el ID de la cuenta y el nombre del rol. No son credenciales, pero en un repositorio público aparecen en los logs de Actions. En las evidencias de este repositorio se han ocultado.

4. Conclusiones

4.1 Parte 1

Terraform desplegó y verificó en Floci seis usuarios, tres grupos, siete pertenencias y tres políticas inline asociadas solo a grupos. El simulador confirmó permisos de desarrollo, operaciones y auditoría, así como denegaciones implícitas para acciones ajenas a cada función.

El caso de `alicia` demuestra la precedencia del `Deny` explícito: recibe permisos de Dev-Team y SecOps-Team, pero no puede borrar objetos S3 según la simulación. La gobernanza por grupos permite mantener y auditar esos permisos sin configuraciones individuales.

El reinicio sin persistencia borró las seis identidades. Terraform detectó y recreó los 18 recursos declarados, lo que confirma que el entorno puede reconstruirse desde código y utilizarse como base limpia para pruebas CI/CD. El simulador verifica decisiones de autorización; una comprobación en AWS real exigiría además revisar ARNs, dependencias entre servicios y restricciones adicionales del entorno de producción.

4.2 Parte 2

El despliegue de la web confirma la misma idea desde otro ángulo: un emulador sirve para probar el código, no la cuenta. Floci aceptó el nombre `nextgen` y sirvió una web privada; AWS rechazó el nombre por ser global y devolvió 403 hasta que se concedió la lectura pública. Learner Lab añadió restricciones propias, como la SCP sobre object lock y la política `voc-cancel-cred`, que ningún emulador local puede anticipar.

Aun así, la prueba en Floci fue la que protegió producción. El commit que eliminaba la portada se detuvo en el primer trabajo, el despliegue no se ejecutó y la web pública siguió funcionando. Que las validaciones estén en el módulo de Terraform permitió que también fallara el workflow `CI`: son dos barreras independientes con el mismo código.

Llevar el kit a Terraform tuvo un coste: estado remoto para los runners efímeros y crear el bucket fuera del proveedor por la SCP. A cambio, el despliegue es declarativo, se puede revisar con `plan` y es idéntico en el PC, en GitHub Actions y en AWS. Solo cambian el endpoint, las credenciales y el estado, y las credenciales nunca salen del secret.